

ÎNDRUMAR DE LABORATOR / LAB GUIDE

LABORATORUL 00 · METODOLOGIE

Metodologia de lucru și template-ul de proiect

Programare Web / Dezvoltarea Aplicațiilor Web

Mediul de lucru, Git și GitHub, template-ul de proiect, laboratoare, proiect, evaluare

Lab methodology and project template

Tehnologia Informației, anul IV · Informatică, anul II

Anul universitar 2026–2027, semestrul I



Ș.l. dr. ing. Eusebiu Pruteanu

Titularul disciplinei



Asist. univ. drd. asoc. Cezar-Marian Papară

Laborator

TITULAR · LABORATOR

Echipa disciplinei și contactul

Course team and contact

Element *Item*	Detalii *Details*	
Titular disciplină *Course lecturer*	**Ș.l. dr. ing. Eusebiu Pruteanu**, Facultatea de Inginerie	
E-mail	pruteanue@ub.ro (subiectul începe cu „[PW]”) · pruteanue@yahoo.com	
Platforma disciplinei *Course platform*	examene.pe.ub.ro (Moodle: materiale, teme, teste)	
Pagina personală *Personal page*	pruteanue.ub.ro	
Birou · Laboratoare	AP20, corpul A, etajul 1 · AP19–AP21, corpul A, etajul 1	
Consultații *Office hours*	miercuri, 09:00–11:00, cu programare prin e-mail	
Laborator *Lab instructor*	**Asist. univ. drd. asoc. Cezar-Marian Papară**, Facultatea de Inginerie	
Profil profesional	Cezar-Marian Papară · Software Developer | PhD & CS University Assistant	
Despre	Software Engineer with 7+ years in IT, architecting scalable enterprise backends, microservices & CI/CD. Research Areas: AI, Smart Logistics, Combinatorial Optimization, Operations Research.	
Comunicare *Contact*	grupul Microsoft Teams al disciplinei	

„Nu sunt cel mai bun, dar mă străduiesc, pentru ca studenții mei să fie.” — E. Pruteanu

CERCETARE

Direcții de cercetare ale echipei

Research areas

Ș.l. dr. ing. Eusebiu Pruteanu

- **Sisteme multi-agent**
  Agenți construiți pe modele de limbaj; agenți web inteligenți.
- **Interacțiunea om-robot și interfețele om-mașină**
  Human-robot interaction and human-machine interfaces.
- **Conducere fuzzy, măsurare și control**
  Sisteme inteligente în automatică.
- **Securitate și IoT**
  Laboratorul UVAB AI Lab, Facultatea de Inginerie.

Asist. univ. drd. asoc. Cezar-Marian Papară

- **Inteligență artificială**
  Artificial Intelligence
- **Logistică inteligentă**
  Smart Logistics
- **Optimizare combinatorie**
  Combinatorial Optimization
- **Cercetări operaționale**
  Operations Research

Programare Web / Dezvoltarea Aplicațiilor Web
Tehnologia Informației, anul IV · Informatică, anul II · Anul universitar 2026–2027, semestrul I

METODOLOGIE DE LUCRU ȘI TEMPLATE DE PROIECT

Cum lucrăm la laborator și cum realizăm proiectul

Completare practică la cursul și platforma disciplinei, coordonate de dl Ș.l. dr. ing. Eusebiu Pruteanu: mediul de lucru, template-ul de proiect, regulile de lucru și proiectul final.

Disciplina	Programare Web (TI, anul IV) / Dezvoltarea Aplicațiilor Web (Informatică, anul II) – titular: Ș.l. dr. ing. Eusebiu Pruteanu
Platforma disciplinei	examene.pe.ub.ro (cursuri, planul săptămânal, teme, test, încărcări) · pagina titularului: pruteanue.ub.ro
Template de proiect	PHP Symfony (backend) + React (frontend) + PostgreSQL, în Docker · github.com/cezarpapara/daw-template-ti-info-2026-2027
Pe scurt	12 laboratoare · proiect individual sau în echipă de maximum 2 · lucrul pe GitHub · susținere fizică

1. Despre acest document

Conținutul teoretic al disciplinei, planul săptămânal, tematica proiectelor și evaluarea sunt stabilite de titularul disciplinei și publicate pe platforma examene.pe.ub.ro. Documentul de față le **aplică la laborator**: descrie ce programe instalăm, cum pornim proiectul, cum lucrăm la fiecare oră și cum realizăm proiectul final.

Este scris pentru studenți fără experiență anterioară: fiecare comandă este însoțită de **locul** în care se rulează, de **motivul** pentru care o rulăm și de **rezultatul** pe care ar trebui să-l vedeți. Capitolele 3–5 se parcurg o singură dată, la început; capitolul 6 descrie lucrul de la fiecare laborator; restul se consultă la nevoie.

2. Ce construim și cu ce

Învățăm să construim o aplicație web **așa cum se face astăzi în firmele de software**: nu un singur fișier care face totul, ci un sistem din mai multe părți, fiecare cu rolul ei, care comunică prin reguli clare [1]. Lucrăm pe un proiect concret, un **magazin online simplu**, cu categorii și produse, pe care îl extindem de la un laborator la altul.

Noțiune	Pe scurt
Backend	programul care rulează pe server: primește cereri, citește și scrie în baza de date, trimite răspunsuri. Al nostru este scris în PHP, cu framework-ul Symfony.
Frontend	aplicația din browser: pagini, butoane, formulare. Nu păstrează date; le cere de la backend. A noastră este scrisă în JavaScript, cu React.
API	„limba” în care vorbesc frontend-ul și backend-ul: o listă de adrese (de exemplu `/api/products`) care răspund cu date în format JSON.
Baza de date	locul în care se păstrează datele, în tabele. Folosim PostgreSQL.
Docker	pornește fiecare componentă într-un „container” izolat, cu tot ce îi trebuie. Proiectul merge identic pe orice calculator.
Git și GitHub	Git ține istoricul modificărilor din cod (*commit*-uri); GitHub păstrează proiectul online, ca să poată lucra mai mulți oameni pe el.

Partea	Tehnologie
Backend	PHP 8.5 + Symfony 8.1, Doctrine ORM, documentația API cu Swagger
Baza de date	PostgreSQL 18
Frontend	React 19 + Vite
Rulare	Docker + Docker Compose (Nginx, PHP, PostgreSQL, React)
Versionare	Git + GitHub

3. Template-ul de proiect

Ca să nu pierdeți timp cu configurări, porniți de la un **template**: un proiect deja configurat și funcțional, publicat la github.com/cezarpapara/daw-template-ti-info-2026-2027. Conține backend-ul (tabelele `category` și `product`, cu date de exemplu, și un API pentru listare, adăugare și ștergere), pagina de documentație a API-ului (Swagger), frontend-ul React (paginile Acasă, Categorii, Produse, Detalii produs), configurarea Docker și o colecție de cereri pentru Postman.

```
Browser ──► FRONTEND  React           (container daw-react,    http://localhost:5173)
              │  cereri API (JSON)
              ▼
            BACKEND   Nginx           (container daw-nginx,    http://localhost:8080)
              │
              ▼       PHP + Symfony   (container daw-php)
              │
              ▼       PostgreSQL      (container daw-postgres, localhost:5432)
```

4. Programe de instalat

Program	La ce îl folosim	Descărcare
Cont GitHub	păstrarea codului și lucrul în echipă; username de forma `prenume-nume`	github.com/signup
Docker Desktop	pornește containerele proiectului; trebuie să fie **pornit** când lucrați	docker.com
Git (cu Git Bash pe Windows)	descărcarea proiectului, commit, push, preluarea noutăților	git-scm.com
Editor de cod	VS Code (gratuit) sau PhpStorm / WebStorm (gratuite pentru studenți)	code.visualstudio.com · jetbrains.com/academy/student-pack
DBeaver Community	vizualizarea tabelelor și rularea interogărilor SQL	dbeaver.io
Postman	testarea API-ului fără să scrieți cod	postman.com

Docker pe Windows. Înainte de instalare, virtualizarea trebuie să fie activă (Task Manager → Performance → CPU → *Virtualization: Enabled*). Pe Windows Pro / Education folosiți **Hyper-V**; pe Windows Home, **WSL 2** (`wsl --install` într-un terminal deschis ca administrator, apoi repornire). Detalii: docs.docker.com.

Verificați instalarea într-un terminal (pe Windows **Git Bash**, pe macOS **Terminal**):

```
docker --version
docker compose version
git --version
```

> **Rezultat.** Fiecare comandă afișează o versiune. Dacă apare `command not found`, programul nu este instalat corect (sau, pentru Docker, nu este pornit).

5. Configurarea inițială (o singură dată)

> **Regula proiectului.** Toate comenzile se rulează din **folderul principal al proiectului** (cel în care `ls` afișează `README.md`, `backend`, `frontend` și `postman`), în terminalul din editor. Excepții: configurarea Git (oriunde) și clonarea (din folderul `proiecte`).

5.1 Configurarea Git

```
git config --global user.name "Ion Popescu"
git config --global user.email "ion.popescu@exemplu.com"
git config --global pull.rebase false
```

Folosiți adresa de e-mail a contului de GitHub: fiecare commit va purta numele vostru. A treia comandă stabilește modul în care Git combină modificările la `git pull`, ca să nu vă întrebe de fiecare dată.

5.2 Clonarea proiectului

5.2.a Creați un folder pentru proiecte

Alegeți un loc simplu pe disc, fără spații și fără diacritice în cale, care nu este pe Desktop și nici într-un folder sincronizat (OneDrive, iCloud). Recomandăm un folder numit `proiecte`.

- **Cu mouse-ul, pe Windows:** deschideți **File Explorer**, intrați în `C:\` (sau în **Documente**), click dreapta pe un spațiu liber → **Nou → Folder** (*New → Folder*) și numiți-l `proiecte`.
- **Cu mouse-ul, pe macOS:** deschideți **Finder**, intrați în folderul personal (cel cu numele vostru), click dreapta → **New Folder** și numiți-l `proiecte`.
- **Din terminal:** pe Windows, în Git Bash: `mkdir -p /c/proiecte`; pe macOS: `mkdir -p ~/proiecte`. Opțiunea `-p` nu dă eroare dacă folderul există deja.

5.2.b Deschideți un terminal în folderul proiecte

- **Windows:** în File Explorer intrați în folderul `proiecte`, click dreapta pe un spațiu liber → **Open Git Bash here** (pe Windows 11, mai întâi **Show more options**).
- **macOS:** în Finder, click dreapta pe folderul `proiecte` → **Services → New Terminal at Folder**. Dacă opțiunea lipsește, o activați din **System Settings → Keyboard → Keyboard Shortcuts → Services → Files and Folders**, sau deschideți aplicația **Terminal** și scrieți `cd ~/proiecte`.

Verificați locul în care vă aflați cu `pwd`: calea afișată trebuie să se termine cu `/proiecte`.

5.2.c Clonați proiectul

```
git clone https://github.com/cezarpapara/daw-template-ti-info-2026-2027.git
```

> **De ce.** „A clona” înseamnă a descărca proiectul de pe GitHub pe calculator, împreună cu tot istoricul lui. Comanda creează în folderul curent un folder nou, `daw-template-ti-info-2026-2027`, cu tot conținutul proiectului. Clonarea se face **o singură dată**; dacă o repetați în același loc, primiți eroarea `destination path already exists`.

5.2.d Verificați că proiectul s-a copiat complet

Deschideți folderul `proiecte/daw-template-ti-info-2026-2027` în File Explorer sau Finder și comparați-l cu pagina proiectului de pe GitHub (github.com/cezarpapara/daw-template-ti-info-2026-2027): trebuie să vedeți aceleași fișiere și foldere, adică `README.md`, `backend`, `frontend` și `postman`. Din terminal, același lucru: `cd daw-template-ti-info-2026-2027`, apoi `ls`.

> **Dacă apare o eroare.** Dacă la clonare sau la verificare apare un mesaj de eroare, nu insistați la întâmplare: faceți o captură de ecran și anunțați cadrul didactic, care vă va îndruma.

5.2.e Deschideți proiectul în editor

Deschideți în editor **exact** folderul `daw-template-ti-info-2026-2027`: nu folderul `proiecte` (un nivel mai sus) și nici `backend` sau `frontend` (un nivel mai jos). Calea arată, de exemplu, așa:

```
Windows:  C:\proiecte\daw-template-ti-info-2026-2027
macOS:    /Users/numele-vostru/proiecte/daw-template-ti-info-2026-2027
```

- **Visual Studio Code** (gratuit): **File → Open Folder…**, alegeți folderul și confirmați **Trust the authors** dacă vi se cere.
- **PhpStorm** sau **WebStorm** (gratuite pentru studenți): **File → Open…**, selectați folderul și apăsați **Open**, apoi **Trust Project**.

5.2.f Terminalul din editor

De aici înainte folosim terminalul integrat în editor:

- **VS Code:** meniul **Terminal → New Terminal** (sau **Ctrl + `**). Pe Windows, din săgeata de lângă **+** alegeți **Git Bash**.
- **PhpStorm / WebStorm:** meniul **View → Tool Windows → Terminal** (sau **Alt + F12**, pe macOS **⌥ F12**), ori tab-ul **Terminal** din partea de jos a ferestrei.

> **Avantajul terminalului din editor.** Se deschide direct în **rădăcina proiectului** (*root*), adică exact în folderul din care se rulează toate comenzile din acest îndrumar. Cât timp nu vă mutați cu `cd` în alt folder, sunteți în locul potrivit; verificați oricând cu `pwd`. Dacă v-ați mutat, reveniți cu `cd` sau deschideți un terminal nou.

5.3 Branch-ul personal

5.3.a Creați branch-ul personal

Branch-ul `main` este copia principală a proiectului, pe care lucrează doar cadrul didactic. Fiecare student lucrează pe branch-ul lui, numit `nume.prenume` (litere mici, fără diacritice, cu punct între nume și prenume). Înlocuiți `popescu.ion` cu numele vostru:

```
git checkout -b popescu.ion        # sau, echivalent: git switch -c popescu.ion
```

`git checkout -b` **creează** branch-ul și vă **mută** pe el. Comanda se rulează o singură dată; a doua oară primiți eroarea `already exists`.

5.3.b Primul push: legarea branch-ului de GitHub

```
git push -u origin popescu.ion
```

- `origin` este numele sub care Git ține minte adresa proiectului de pe GitHub;
- `popescu.ion` este branch-ul pe care îl trimiteți;
- `-u` (prescurtarea lui `--set-upstream`) **leagă o singură dată** branch-ul local de cel de pe GitHub. După acest prim push, comenzile `git push` și `git pull`, scrise fără alți parametri, știu singure unde să trimită și de unde să aducă modificările.

> **Forma completă și forma scurtă.** Mai departe puteți folosi oricând forma completă, `git push origin popescu.ion`, sau forma scurtă, `git push`. Forma scurtă funcționează **numai** dacă ați făcut anterior, o dată, `git push -u origin popescu.ion`; altfel Git răspunde cu `has no upstream branch`.

> **Rezultat.** La primul push se deschide autentificarea GitHub în browser. `git branch` afișează `* popescu.ion`, iar pe GitHub branch-ul vostru apare în lista de branch-uri. Dacă primiți eroarea **403** (*Permission denied*), trimiteți cadrului didactic username-ul de GitHub; după ce acceptați invitația primită pe e-mail, repetați `git push -u origin popescu.ion`.

5.4 Prima pornire

Docker Desktop trebuie să fie pornit. Rulați comenzile **una câte una**, în terminalul din editor, așteptând să se termine fiecare:

Comanda	Ce face	Când se rulează din nou
`docker compose -f backend/compose.yaml up -d --build`	construiește imaginile și pornește containerele backend-ului (Nginx, PHP, PostgreSQL); prima dată durează câteva minute	fără `--build` la fiecare pornire; cu `--build` doar dacă se modifică fișierele din `backend/docker`
`docker exec -it daw-php composer install`	instalează bibliotecile PHP ale proiectului (Symfony, Doctrine etc.) în `backend/app/vendor`	doar prima dată și când `composer.json` se schimbă (de exemplu după preluarea noutăților de pe `main`)
`docker exec -it daw-php php bin/console doctrine:database:create --if-not-exists`	creează baza de date `daw`	o singură dată (repetată nu strică nimic, dar nu mai este necesară)
`docker exec -it daw-php php bin/console doctrine:migrations:migrate`	creează tabelele și datele de exemplu; la întrebare răspundeți `yes`	doar când apar migrări noi (ale voastre sau de pe `main`); fără migrări noi afișează *Already at the latest version*
`docker compose -f frontend/compose.yaml up -d --build`	construiește și pornește containerul React (instalează pachetele JavaScript în container)	fără `--build` la fiecare pornire; cu `--build` doar dacă se modifică fișierele din `frontend/docker`

> **Atenție.** Nu repetați clonarea și nici crearea branch-ului: ele se fac o singură dată. La orele următoare porniți proiectul după pașii din „Începutul fiecărui laborator”.

> **Rezultat.** localhost:8080/api/health afișează „Salut din Symfony!”, localhost:8080/api/doc afișează documentația API (Swagger), iar localhost:5173 deschide aplicația, cu o casetă verde pe pagina Acasă.

6. Metodologia de lucru la fiecare laborator

6.1 Începutul fiecărui laborator

1. Porniți **Docker Desktop** și așteptați indicația *Engine running*.
2. Deschideți proiectul în editor și terminalul integrat (sunteți automat în rădăcina proiectului).
3. Verificați pe ce branch sunteți:

```
git branch          # ramura curenta are * in fata
git status          # primul rand: On branch ...
```

4. Doar dacă **nu** sunteți deja pe branch-ul vostru, treceți pe el (dacă sunteți, săriți acest pas):

```
git checkout popescu.ion        # sau: git switch popescu.ion
```

5. Porniți containerele (fără `--build`):

```
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
```

6.2 În timpul lucrului

Modificările din cod se văd imediat, fără repornire: codul backend-ului este în `backend/app/src/`, iar al frontend-ului în `frontend/app/src/`.

6.3 Salvarea lucrului pe GitHub

De câte ori terminați ceva (o pagină, o funcționalitate), faceți un commit mic, cu un mesaj clar, și îl trimiteți pe GitHub:

```
git status
git add .
git commit -m "Laboratorul 04: pagina Despre"
git push origin popescu.ion     # sau doar: git push (dupa git push -u)
```

`git status` arată ce s-a modificat; `git add .` pregătește modificările; `git commit -m "…"` le salvează local, cu un mesaj scurt care spune **ce** ați făcut; `git push` le trimite pe GitHub, pe branch-ul vostru.

6.4 Sfârșitul laboratorului

```
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

Containerele se opresc; codul și datele din baza de date rămân.

6.5 Preluarea noutăților de pe main

Când cadrul didactic anunță noutăți pe `main`, salvați întâi lucrul vostru (commit și push), apoi:

```
git checkout main                # sau: git switch main
git pull origin main
git checkout popescu.ion
git merge main --no-edit
git push origin popescu.ion      # sau doar: git push
```

Dacă apare `CONFLICT`, deschideți fișierul indicat, alegeți în editor varianta corectă (**Accept Current / Incoming / Both**), eliminați marcajele `<<<<<<<`, `=======`, `>>>>>>>`, apoi `git add .`, `git commit --no-edit` și `git push origin popescu.ion`. Nu se pierde nimic; la nevoie, `git merge --abort` anulează operația.

7. Baza de date și API-ul

DBeaver: Database → New Database Connection → PostgreSQL, cu `localhost`, portul `5432`, baza, utilizatorul și parola `daw`. Tabelele apar la *daw* → *Schemas* → *public* → *Tables*. **Swagger:** localhost:8080/api/doc, butoanele *Try it out* și *Execute*. **Postman:** Import → `postman/DAW-API.postman_collection.json`.

Metodă	Adresă	Ce face
GET	`/api/health`	verifică backend-ul și baza de date
GET / POST	`/api/categories`	listează / adaugă categorii
GET / POST	`/api/products`	listează / adaugă produse
GET / DELETE	`/api/products/{id}`	afișează / șterge un produs

8. Planul laboratoarelor

Laboratoarele urmează temele din planul săptămânal al disciplinei, publicat de titular pe platformă, aplicate pe template-ul de proiect.

Lab.	Titlu	Ce adăugăm în proiect
01	Mediul de lucru și template-ul de proiect	proiectul pornit, branch-ul personal, primul commit
02	HTML5 și CSS3: semantică, Flexbox, Grid, responsiv	o pagină de prezentare, adaptată pentru mobil

Lab.	Titlu	Ce adăugăm în proiect
03	JavaScript: DOM, evenimente, fetch	interactivitate și date dintr-un API public
04	React: componente, JSX și props	pagina „Despre”, componenta `Card`
05	React: stare, evenimente și formulare	formular de contact, căutare în produse
06	Symfony: rute, controllere, JSON	primele endpoint-uri proprii
07	Baza de date: entități, relații, migrări	entitatea `Review` (recenzii la produse)
08	API REST complet (CRUD)	toate operațiile pentru recenzii
09	Legarea interfeței React de API	recenziile afișate și adăugate din interfață
10	Validare și securitate de bază	validări în backend, erori afișate în interfață
11	Lucrul în echipă cu Git și depanarea	branch de echipă, conflicte, DevTools, loguri
12	Calitate, performanță, pregătirea proiectului	audit Lighthouse, README, verificarea finală

9. Proiectul

9.1 Ce realizați

Proiectul este **o aplicație web funcțională pe o temă la alegere**, construită cu tehnologiile din template. Scopul este să demonstrați că puteți structura o aplicație completă, cu front-end și back-end, că aplicați bunele practici și că puteți explica tot ce ați făcut.

- **Individual sau în echipă de maximum 2 studenți.** Contribuția fiecăruia trebuie să reiasă clar din commit-urile de pe GitHub.
- **Tema** se alege din lista de proiecte de pe platforma disciplinei (secțiunea „Pentru realizare Proiect”) sau se propune, cu aprobarea cadrului didactic. Exemple: platformă de rețete, bibliotecă (gestiunea cărților), rezervări la evenimente, buget personal cu grafice, aplicație de quiz, kanban pentru sarcini, platformă de feedback pentru studenți, site pentru o cafenea cu rezervări, platformă de voluntariat, aplicație de fitness.
- **Unde lucrați:** individual, pe branch-ul vostru `nume.prenume`; în echipă, pe un branch comun `proiect-nume1-nume2` (laboratorul 11).

9.2 Cerințe minime

1. cel puțin **două entități proprii**, legate printr-o relație (de exemplu *Rețetă – Ingredient*), cu migrări;
2. un **API REST** cu operațiile CRUD pentru entitatea principală, cu validarea datelor și coduri de stare corecte;
3. **pagini React** pentru listare, detalii, adăugare și ștergere, cu mesaje de încărcare și de eroare;
4. o interfață **responsivă**, ușor de folosit (aplicați ce ați învățat la IOC);
5. un **README** cu descrierea aplicației, tehnologiile, instrucțiunile de rulare, capturi de ecran și contribuția fiecărui membru;
6. un istoric Git cu **commit-uri regulate**, mici și cu mesaje clare (nu un singur commit la final).

9.3 Documentația

Pe lângă README, documentația proiectului se realizează după **îndrumarul și template-ul de proiect al disciplinei**, de pe platformă (*Îndrumar proiect – template PSW + PWEB*): scopul aplicației, cerințele, arhitectura, modelul datelor, interfața (cu capturi și scenarii de utilizare), testarea, concluziile și bibliografia.

9.4 Predare și susținere

- **Predarea:** codul se află pe GitHub, pe branch-ul vostru (sau al echipei); proiectele de pe GitHub nu mai trebuie încărcate separat. Documentația se încarcă pe examene.pe.ub.ro, la „Upload Proiecte/Teme/Laboratoare/Referate”.
- **Susținerea este fizică**, cu demonstrația aplicației și întrebări; în echipă, ambii membri prezintă și răspund.
- **Când:** în ultimele întâlniri de curs sau laborator, când sunt prezenți atât titularul, cât și cadrul didactic de la laborator, sau în ziua examenului. Programarea exactă se stabilește pe parcurs și se anunță pe platformă și pe Teams.
- După susținere, completați **autoevaluarea** de pe platforma disciplinei („AutoApreciere realizare proiect”).

9.5 Evaluarea

Nota finală are trei componente, conform regulilor stabilite de titularul disciplinei: **nota de laborator** (activitatea și prezența la laborator), **nota pe proiectul final** și **nota la testul teoretic** de pe platformă. Proiectul se apreciază după criteriile publicate de titular:

Criteriu	Punctaj maxim	Ce se urmărește
Structura și funcționalitatea aplicației	30 p	aplicație completă, navigare logică, funcții corecte
Design și interfață (UI/UX)	20 p	aranjare coerentă, responsivă, estetică
Cod și bune practici	20 p	cod curat, comentat, bine organizat
Inovație și complexitate	15 p	funcționalități originale sau avansate
Documentație și prezentare	15 p	README clar, explicații, demonstrație

> **Integritate academică.** Instrumentele de inteligență artificială pot fi folosite ca asistent, dar trebuie să puteți explica fiecare linie de cod pe care o prezentați. Codul preluat din alte surse se marchează în comentarii, cu sursa. Partea pe care nu o puteți explica la susținere nu se punctează.

10. Probleme frecvente

Mesajul sau simptomul	Cauza și rezolvarea
`fatal: not a git repository`	terminalul nu este în folderul principal; verificați cu `pwd`
`no configuration file provided`	comanda `docker compose` nu este rulată din folderul principal
`Cannot connect to the Docker daemon`	Docker Desktop nu este pornit
`No such container: daw-php`	backend-ul nu este pornit (capitolul 6)
`composer: command not found`	lipsește prefixul `docker exec -it daw-php`
`port is already allocated`	alt program folosește portul 8080, 5173 sau 5432; închideți-l
`relation "product" does not exist`	nu s-au rulat migrările (capitolul 5, pasul 4)
caseta roșie pe pagina Acasă	backend-ul nu este pornit sau nu răspunde la `/api/health`
`403` la `git push`	nu aveți încă acces la proiect; trimiteți username-ul de GitHub
`the input device is not a TTY`	în Git Bash separat pe Windows: rulați din terminalul editorului sau adăugați `winpty` în față

Pentru orice altă eroare de backend, citiți ultimele linii din `backend/app/var/log/dev.log` sau rulați `docker compose -f backend/compose.yaml logs -f` (ieșire cu **Ctrl + C**).

11. Resurse pentru exersare

- **W3Schools – HTML / CSS / JavaScript:** https://www.w3schools.com/html/
- **W3Schools – React:** https://www.w3schools.com/react/
- **W3Schools – PHP:** https://www.w3schools.com/php/
- **W3Schools – SQL:** https://www.w3schools.com/sql/
- **W3Schools – Git:** https://www.w3schools.com/git/
- **Curs HTML și CSS în limba română (playlist):** https://www.youtube.com/playlist?list=PL7aWL1nkGk0zssBjfDBtRMxtAGL3KNmAJ
- **Curs JavaScript în limba română (playlist):** https://www.youtube.com/playlist?list=PL7aWL1nkGk0ytE-fV1pApIdKI2ZwIyw-5
- **React, în limba română (playlist):** https://www.youtube.com/playlist?list=PLZuysB_DjFARX5kWZGu8hvBbmw-3VWd8w
- **Git, în limba română:** https://www.youtube.com/watch?v=iVpuVkVeBcw
- **Pro Git, traducere în limba română:** https://git-scm.com/book/ro/v2
- **Docker, introducere (engleză):** https://www.youtube.com/watch?v=pTFZFxd4hOI
- **Symfony, curs complet (engleză):** https://www.youtube.com/watch?v=Bo0guUbL5uo
- **Documentație: Symfony:** https://symfony.com/doc/current/index.html
- **Documentație: React:** https://react.dev/learn
- **Documentație: Docker:** https://docs.docker.com/get-started/
- **Documentație: PostgreSQL:** https://www.postgresql.org/docs/current/
- **Platforma disciplinei (curs, plan săptămânal, teme, bibliografie):** http://examene.pe.ub.ro/

Bibliografie

[1] E. Pruteanu și I. Furdu, *Modele, tehnologii, servicii și aplicații integrate de tip R.I.A. pentru web*. Bacău: Alma Mater, 2022.

[2] E. Pruteanu, *Programare / Dezvoltare web*, curs și plan săptămânal, Universitatea „Vasile Alecsandri” din Bacău, 2026. Disponibil pe examene.pe.ub.ro.

[3] S. Buraga, *Tehnologii Web*. București: Matrix Rom, 2001.

[4] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, teză de doctorat, University of California, Irvine, 2000.

[5] S. Chacon și B. Straub, *Pro Git*, ed. a 2-a. Apress, 2014. Traducere în română: git-scm.com/book/ro/v2

[6] Symfony, *The Symfony Documentation*. symfony.com/doc

[7] Meta Open Source, *React Documentation*. react.dev

[8] Docker Inc., *Docker Documentation*. docs.docker.com

[9] Mozilla, *MDN Web Docs*. developer.mozilla.org